

IIITA Timetabling System

*A thesis submitted in partial fulfillment of the requirements
for the award of the degree of*

BACHELOR OF TECHNOLOGY



By:-

ESHANT

RAHUL ROY

LAKAVATH PEER SINGH

Enrollment No.

IIT2023198

IIT2023196

IIT2023197

to the

DEPARTMENT OF INFORMATION TECHNOLOGY

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
ALLAHABAD

Wednesday 7th May, 2026



भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad

An Institute of National Importance by Act of Parliament

Deoghat Jhalwa, Prayagraj 211015 (U.P.) India

Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CERTIFICATE

It is certified that the work contained in the thesis titled "IIITA Timetabling System" by Eshant, Rahul Roy, Lakavath Peer Singh has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Department of Information Technology
IIIT Allahabad



भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad

An Institute of National Importance by Act of Parliament

Deoghat Jhalwa, Prayagraj 211015 (U.P.) India

Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CANDIDATE DECLARATION

We, Eshant (IIT2023198), Rahul Roy (IIT2023196), and Lakavath Peer Singh (IIT2023197), hereby certify that the thesis titled “IIITA Timetabling System” has been submitted by our in partial fulfillment of the requirements for the Degree of Bachelor of Technology in the Department of Information Technology at the Indian Institute of Information Technology, Allahabad.

We acknowledge that plagiarism includes:

1. Reproducing another person's work (fully or partially) or ideas and presenting them as our own.
2. Copying or paraphrasing another person's work without appropriate attribution.
3. Engaging in literary theft by reproducing a unique literary construct without crediting its source.

We affirm that due credit has been given to all original authors and sources through proper citations for words, ideas, diagrams, graphics, computer programs, experiments, results, and websites that are not our own contributions. Direct quotations have been appropriately marked, and sources have been properly referenced. Furthermore, we declare that no portion of our work is plagiarized. In the event of a plagiarism claim, we accept full responsibility for the contents of this thesis. We acknowledge that our supervisor may not be able to verify the originality of every aspect of our work.

Place: **IIITA**

Date: **07/05/2026**

Eshant (IIT2023198)

Rahul Roy (IIT2023196)

Lakavath Peer Singh (IIT2023197)

IIITA Timetabling System

ABSTRACT

University timetabling is a complex problem, or rather, it is an NP-hard problem. Even small campuses have hundreds of constraints to schedule for: courses, professors, rooms, and groups of students. At IIIT Allahabad, the timetables were generated manually in spreadsheets, which was time-consuming, error-prone, and not scalable. In this thesis, we created the "IIITA Timetabling System", a web-based solution to automate and digitise the entire timetable scheduling pipeline.

We implemented a heuristic-based Excel parser that can process raw, unstructured timetable Excel spreadsheets, extracting information such as subject codes, faculty names and room numbers, and loads them into a cloud-based PostgreSQL database using Supabase.

The main contribution is an automated timetable scheduler based on a (1+1) Evolution Strategy (ES). We represent a timetable as a chromosome with Gene decisions (day, time-slot, room) and evaluate candidate solutions using a fitness function with 10 hard constraints (room / professor / group clashes, break boundary limits, labs, elective synchronisation, etc.) and 3 soft constraints (minimising gaps for students, maximising room occupancy and keeping professors in one side of the day). The algorithm adjusts step sizes using Rechenberg's classic 1/5th rule, restarts when trapped in local optimums and can be initialised with a given timetable. To prevent clashes across semesters, we populate already scheduled sessions from published timetables.

Since no solver can anticipate every conceivable real-world preference, we also provide a drag-and-drop Timetable Editor. Administrators can visually rearrange the sessions, with conflicts displayed in real-time, and LNS (Large Neighbourhood Search) auto-fix with one click. A Master View displays slots from all published timetables so conflicts across semesters are instantly apparent.

In daily life, however, the system allows customized views for each stakeholder, from the students' view of their individual schedule to professors' schedules and a master occupancy map for the entire university campus, plus the ability to find available rooms using our Free Room Finder module. In addition, we developed an extensive Intelligent Generator to guide administrators step-by-step.

React 19, TypeScript, and Tailwind CSS form the technology stack used throughout the application, making it very flexible and modern in design. The use of the Google Sheets API makes data export elegant, while PDF creation is possible locally as well. In the context of this thesis, topics such as the theoretical framework of the timetabling problem, the technology stack and architecture, evolutionary algorithm with constraints, and interactive editor components are discussed.

Contents

Abstract	iii
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives of the Study	3
1.4 Scope and Limitations	5
1.5 Organization of the Thesis	5
2 Literature Review	7
2.1 The University Timetabling Problem (UTP)	7
2.1.1 Constraint Satisfaction and Complexity	7
2.2 Algorithmic Approaches to Timetabling	9
2.2.1 Exact Methods	9
2.2.2 Heuristic and Meta-Heuristic Algorithms	9
2.3 Evolution of Timetable Management Systems	10
2.3.1 Legacy Systems: Spreadsheets and Desktop Applications	10
2.3.2 Modern Web-Based Systems	11
2.4 Identification of Gaps in Existing Solutions	12
3 Technologies and Frameworks	14
3.1 Frontend Architecture: React 19 and TypeScript	14
3.1.1 Performance and Data Reliability	15
3.2 Construction and Styling: Vite and Tailwind CSS	15
3.3 Back-end and Data Ingestion: Supabase	16
3.4 Solver Engine: Architecture of Pure TypeScript	16
3.5 External Integration: Reporting and Exporting	17
4 System Architecture and Design	18
4.1 Application Topology	18
4.2 Database Schema Design	19
4.2.1 Core Entities	19
4.2.2 The Central Hub: timetable_slots	20
4.2.3 Solver Data Structures	21
4.3 Asynchronous Processing Design	22
5 Implementation and Results	25
5.1 Introduction to the Optimization Engine	25
5.2 Mathematical Representation of the Schedule	25

5.3	Formulation of Hard/Soft Constraints	27
5.3.1	Hard Constraints (Feasibility Space)	27
5.3.2	Soft Constraints (The Quality Space)	29
5.4	The (1+1) Evolution Strategy Solver	30
5.4.1	Strict Elitism	30
5.4.2	Mutation Operators	31
5.4.3	Adaptive Variance (The Brain)	31
5.4.4	Stagnation Recovery	32
5.5	Interactive Editor & Large Neighbourhood Search (LNS)	32
5.6	Results and Application Features	35
5.6.1	Algorithmic Convergence	35
5.6.2	The Interactive Drag-and-Drop Grid	35
5.6.3	LNS Auto-Fix Execution	35
5.6.4	Export and PDF Generation	36
5.7	Dynamic Time Grid Generation	36
5.8	Multi-Dimensional Clash Detection Algorithm	37
5.9	Inverted Logic: The Free Room Finder	38
5.10	React Component Hierarchy	40
5.11	Role-Based Viewer Implementations	40
5.11.1	Student Cohort View	40
5.11.2	Master Occupancy Viewer	41
5.12	Export Approaches and Outcomes	41
5.12.1	Client-Side PDF Export	41
5.12.2	Google Sheets Export	42
5.12.3	User Interface Evaluation	42
5.13	The Interactive Timetable Editor	42
5.13.1	Drag-and-Drop Session Rescheduling	43
5.13.2	Inline Editing	43
5.13.3	Real-Time Conflict Visualization	43
5.13.4	Feasibility Check	44
5.13.5	LNS Auto-Fix	44
6	Conclusion and Future Work	46
6.1	Summary of Contributions	46
6.2	Future Work – Phase 2 – Integration with M.Tech	47
6.2.1	The Basket Tethering Problem	48
6.2.2	Solving Core Duality	48
6.2.3	Meta-Grouping and Collision Handling	48
6.2.4	Scalability of Algorithmic Solution	49
6.2.5	Technical Requirements at the Core	49
6.3	Conclusion	49
	References	50
A	Core Algorithm Snippets	51
A.1	Dynamic Grid Generation Logic	51
A.2	Real-Time Clash Detection Filter	51
A.3	Solver Fitness Evaluation Function	52
A.4	(1+1)-ES Solver Main Loop	53
A.5	LNS Auto-Fix for the Editor	54

List of Figures

3.1	High-Level System Architecture and Component Interaction	15
4.1	Entity-Relationship Diagram of the Complete Database Schema	19
4.2	UML Class Diagram of the Solver Data Model	21
4.3	Data Ingestion Pipeline for Excel Timetables	23
5.1	Sequence Diagram: End-to-End Intelligent Generator Workflow	26
5.2	Activity Diagram: (1+1)-ES Solver Execution Flow	33
5.3	Activity Diagram: LNS Destroy-and-Repair Auto-Fix	34
5.4	Inverted Logic Flowchart for the Free Room Finder Algorithm	39
5.5	Hierarchical Structure of the Core React Components	40
5.6	Activity Diagram: LNS Destroy-and-Repair Auto-Fix	45

List of Tables

2.1	Comparison of Timetabling Methodologies	13
3.1	Summary of the Technology Stack	14
4.1	Schema Specification for <code>timetable_slots</code>	20
5.1	Hard Constraints Enforced by the Solver	29
5.2	Types of Scheduling Clashes Detected	37
5.3	Role-Based Component Mapping and Data Scope	40

Chapter 1

Introduction

Each academic semester, the university faces the same challenging problem. How do you allocate hundreds of classes, a number of lecturers, and a fixed set of class rooms to a semester-wide timetable? This problem is known as the University Timetabling Problem (UTP) and it has been proven NP-hard, i.e., that there is no efficient algorithm for solving the problem optimally. This chapter will give a brief overview of the IITA Timetabling System, describe its rationale, and outline the remaining chapters in this thesis.

1.1 Background and Motivation

An academic timetable involves assigning lectures, tutorials, and laboratory classes to a particular time slot, venue, and lecturer, while taking into account the limitations imposed by the situation. There are some constraints that are rigid, in that it is not possible to double book the lecturer or run two classes in one room at the same time. On the other hand, there are certain soft constraints which include having the minimum amount of gap between lectures, and the professors prefer lecturing in one half of the day only.

IIIT Allahabad is one among the many colleges in India where the scheduling process remains manual. Timetables are created manually using Microsoft Excel, and conflicts are manually resolved by the academic coordinators. Even though Microsoft Excel is highly

customizable and flexible, it does not have any sense of the meaning of a “professor” or “room”. It is up to the manual efforts of the people preparing the timetables to make sure that such problems are avoided.

The issues do not stop even after the timetable is released. Using a static PDF or Excel file will result in the need to send out the whole document every time there is a change, such as a classroom change or an instructor change. The students will have to work with outdated versions, and no one will be able to answer basic queries such as "What classrooms are available at 2 PM on Wednesday?"

These annoyances were what pushed us to develop the IIITA Timetabling System. We sought a unified solution that would become the ultimate authority on all scheduling information. This system should be capable of automatically generating conflict-free timetables, allowing the administrators to adjust them interactively, and providing individualized timetables for each stakeholder.

1.2 Problem Statement

The underlying issue we sought to address is simple: The manual creation and management of timetables via spreadsheets at IIITA is inefficient, error-prone, and inherently inflexible. Specifically, the existing process suffers from:

- **Unstructured Data:** Our schedule is created in excel sheets with merged cells, non-matching names, and relations that are beyond any computer program’s comprehension.
- **Manual Conflict Resolution:** To check whether there are problems like clash of classrooms, unavailability of certain teachers or courses, one needs to do a whole

process manually, hence increasing the possibility of error.

- **No Automated Generation:** Currently, there is no system to automatically create a non-conflict timetable according to the needs of our institution.
- **One-Size-Fits-All Views:** It makes no sense to make a timetable which will not satisfy everyone. Students need to search in this timetable, and teachers cannot quickly find their schedules.
- **Resource Management:** Administrators lack a simple way of knowing which classrooms are vacant at certain times and how effectively our resources are being used.
- **Static Distribution:** After publishing the timetable in pdf form, updating it means distributing it all over again.

Thus, our objective was to develop a system which could help close the chasm between unstructured spreadsheet formats and structured data, schedule creation, editing tools, and provide role-based visualization of the timetable.

1.3 Objectives of the Study

We set ourselves seven concrete objectives:

1. **Heuristic Data Ingestion:** Design an engine that will parse IIITA's complicated Excel timetable, extract metadata (course subjects, instructors, class venues), merge cells, and normalise into a relational schema.
2. **Real-Time Clash Detection:** Design a set of algorithms for detecting conflicts related to time, room availability, instructor workload, and student sections immediately upon importing timetable data.

3. **Role-Based Views:** Design role-based UI interfaces for different actors in the process including:
 - Student Interface with schedule filtering options and electives.
 - Professor Interface with personal schedule only visible to the professor.
 - Admin Interface with master schedule, free room finder, and other administrative tools.
4. **Export and Reports:** Provide users with options to export schedule as PDF, as well as providing Google Sheets support for administrative purposes.
5. **Automated Timetable Generation:** Implement a $(1 + 1)$ Evolution Strategy solver that evaluates candidates against 10 hard constraints and 3 soft constraints, adapts its step size via the $1/5$ th success rule, supports warm-starting from existing schedules, and handling inter-semester constraints through locked sessions.
6. **Interactive Timetable Editor:** Develop a graphical drag-and-drop editor for manual timetable optimisation with instant conflict detection, room and professor modification capabilities, and one-click Large Neighbourhood Search (LNS) auto-resolution for any remaining conflicts.
7. **Intelligent Generator:** Create a guided, multi-step workflow that guides administrators through cluster selection, home room mapping, professor assignment, and solver configuration.

1.4 Scope and Limitations

The scope of the project is to implement all phases of the IIITA Timetabling System, including the React + TypeScript front end, the PostgreSQL back end with Supabase, and the browser-based execution of the (1+1)-ES. Even though the system has been designed specifically for the timetable of IIIT Allahabad, it is still extensible for other colleges.

That said, there are a few important limitations to keep in mind:

- **Parser Sensitivity:** Our Excel parser relies on heuristic rules tuned to IIITA's template. Major deviations from that format may require tweaking the parsing logic.
- **Single-Threaded Solver:** The solver runs on the browser's main thread using batched `setTimeout` calls. This strategy will work well with the present input size, though in case of very large inputs it may become a performance bottleneck.
- **Heuristic Optimality:** Evolution Strategies are meta-heuristics. They guarantee feasibility by providing solutions with zero hard constraint violations, though there is no assurance of optimality for soft constraints based solutions. This depends upon the number of generations.

1.5 Organization of the Thesis

The rest of this thesis is organised as follows:

Chapter 2: Literature Review will provide a review of current researches into university timetabling, including the basic theories of UTP as an NP-hard problem and current algorithmic solutions in literature.

Chapter 3: Technologies and Frameworks will present our choice of technology stack for this project, i.e., React 19, TypeScript, Tailwind CSS, Vite, and Supabase, along with the reasons for their adoption.

Chapter 4: System Architecture and Design will detail the database schema, the data ingest pipeline, the solver architecture, and finally, the client-side routing of components.

Chapter 5: Implementation and Results will form the core of this thesis, where the implementation details of the Excel parser, clash detection algorithms, (1+1)-ES solver and constraint system, drag-and-drop editor and auto-LNS fix, and the intelligent generator will be described.

Chapter 6: Conclusion and Future Work sums up what we achieved, reflects on the challenges we faced, and outlines future directions such as parallelised solvers and hybrid meta-heuristics.

Chapter 2

Literature Review

It is not a new phenomenon that the problem of designing schedules that have no conflicts and that are perfect has received much attention through decades of research. This chapter summarizes the current literature on the UTP, categorizes the different types of algorithms developed by researchers for tackling the problem, and discusses the evolution of software architecture in the context of timetable management systems.

2.1 The University Timetabling Problem (UTP)

Formally speaking, academic scheduling is referred to as the problem of allocating a collection of activities (like lectures, examinations, and meetings) to a collection of resources (time periods, rooms, and teaching staff) based on a variety of constraints [schaerf1999survey]. It should be noted that the difficulty of such a task depends on many factors, such as the scale of the institution, the flexibility of the course structure, and the constraints' strictness.

2.1.1 Constraint Satisfaction and Complexity

In computational complexity theory, the University Timetabling Problem is generally classified as NP-hard [even1975complexity]. This means that when the number of variables

(classes, rooms, teachers) increases, the time needed to solve the problem increases exponentially, making any exhaustive search algorithm totally impractical.

Constraints within the UTP are universally divided into two categories:

- **Hard Constraints:** These are non-negotiable conditions that must be satisfied for a timetable to be considered feasible. Examples are:
 - A room cannot host more than one class simultaneously.
 - A professor cannot teach two different classes at the same time.
 - A student section cannot be scheduled for two different lectures simultaneously.
 - The seating capacity of an assigned room must meet or exceed the number of students enrolled in the class.
- **Soft Constraints:** These represent preferences that enhance the quality of the timetable but are not strictly necessary for its execution. A penalty score is usually assigned when soft constraints are violated, and the goal of optimization algorithms is to minimize this score. Examples include:
 - Minimizing the number of free periods (gaps) in a student's daily schedule.
 - Ensuring faculty members do not have heavily fragmented schedules across the week.
 - Assigning classes to rooms preferred by the instructor.
 - Grouping specific core subjects together sequentially.

2.2 Algorithmic Approaches to Timetabling

Over the years, researchers have proposed a vast array of techniques to solve the UTP, ranging from classical mathematical modeling to modern meta-heuristic algorithms.

2.2.1 Exact Methods

The earliest methods concerned at the mathematical level whether Integer Linear Programming (ILP) and Graph Coloring. In a Graph Coloring model, events are represented as nodes, and edges are drawn between events that conflict (e.g. share the same student group, professor). The objective is to use the fewest possible colours to colour the nodes, where colours indicate distinct time slots, such that no two adjacent nodes are coloured the same colour. Although theoretically valid, exact approaches have significant difficulties due to the high dimension of real-life timetabling problems, which are often unable to find solutions in acceptable computation times when soft constraints are added.

2.2.2 Heuristic and Meta-Heuristic Algorithms

Because exact methods could not deal with large-size problems, heuristics and meta-heuristics became a prominent focus of the scientific community during the late 1990s and 2000s. The methods do not guarantee the absolute best solution, but they are very good at finding “good enough” (feasible and high-quality) solutions in reasonable time.

- **Genetic Algorithms (GA):** Inspired by natural evolution, GAs represent potential timetables as "chromosomes." Through processes of selection, crossover (combining parts of two schedules), and mutation (randomly altering a scheduled event), populations of timetables evolve over generations to minimize constraint violations [burke1996genetic].

- **Simulated Annealing (SA) and Tabu Search:** These local search techniques iteratively make small changes to a complete schedule. Tabu search uses a memory structure (a "tabu list") to prevent the algorithm from revisiting recently evaluated schedules, thereby avoiding getting trapped in local optima [glover1997tabu]. Simulated Annealing mimics the metallurgical process of annealing, occasionally accepting "worse" solutions early in the process to explore a broader search space before gradually "cooling" down to focus on strict optimization [kirkpatrick1983optimization].

2.3 Evolution of Timetable Management Systems

Although a lot of work has been done by researchers to understand schedule generation mathematically, it cannot be ignored that software engineering for timetable management has advanced considerably, in tandem with advances made in software engineering as a whole.

2.3.1 Legacy Systems: Spreadsheets and Desktop Applications

Timetables were traditionally hand-drawn on large whiteboards and grids. The rise of personal computing gave birth to spreadsheet programs such as Microsoft Excel, which became the standard in the academic world. Spreadsheets provide an immensely versatile user interface that is easy to read and understand. As observed by many educational technology experts, however, spreadsheets are inherently static in nature. They are unable to incorporate relational data models that would allow referential integrity enforcement and detection of logical inconsistencies without relying on error-prone, custom-built

macros (e.g., Visual Basic for Applications) [panko1998what]. The problem with sharing an Excel document is the static nature of its data.

Desktop application solutions emerged in the late 1990s and early 2000s as a solution to the drawbacks associated with spreadsheet software. These systems incorporated relational database management systems such as MySQL or Oracle for storing schedule information, providing superior data integrity compared to flat file storage. Nevertheless, due to the desktop-centric nature of such applications, they also had the same distribution problems. The stakeholders involved, namely students and instructors, could not directly communicate with the software but had to resort to printed documents or HTML outputs produced by administrators.

2.3.2 Modern Web-Based Systems

Cloud computing and contemporary web frameworks have greatly influenced the way institutions manage their logistics through software systems. Modern systems employ full-stack web-based architectures, enabling the storage of data centrally while ensuring universal access using web browsers.

With the use of modern front end frameworks such as React, Angular, or Vue.js, highly interactive Single Page Applications (SPAs) have become possible. In contrast to legacy web applications that necessitated full-page refreshes every time an action is performed by a user, SPAs allow for highly fluid user interactions. This is especially important in the case of timetable interfaces since the user needs to be able to change views quickly, for example, switching from "Room view" to "Professor view" and filtering for particular criteria (e.g., only particular students' sections).

In addition, Backend as a Service (BaaS) providers such as Firebase or Supabase have sped up this process considerably. BaaS platforms remove the burden of setting up servers, creating databases, and securing authentications which allows developers to focus on implementing functionality such as conflict detection and heuristic Excel parsers.

2.4 Identification of Gaps in Existing Solutions

While there is an abundance of off-the-shelf schedulers and extensive academic literature on constraints-based scheduling, it seems that there is still a significant gap in many mid-size to large academic institutions, especially those based in developing countries.

1. **The Ingestion Barrier:** Most commercial software applications demand manual data entry via their interface or importing the data from rigidly defined CSV files. Any institution that maintains a more elaborate, unique Excel format (including merged cells, cell coloring, and embedded notes) will face major difficulties converting its current workflow to such a rigid system. There is an urgent need for heuristic parsers that can read "human-written" spreadsheets.

2. **Inadequate Building Occupancy Tools:** Most timetabling applications only produce the desired schedule output (student/faculty). However, no tools are available for campus managers to visualize building occupancy. A "Room Finder" utility or "Master Occupancy Map" is not typically offered as part of the core scheduling engine.

3. **Disconnect between Generation and Management:** The highly scientific solutions provided in academia for scheduling, which are based on Genetic Algorithms, are characterized by terrible interfaces which are not usable by management teams. On the other

hand, well-designed commercial products usually present a fixed schedule that is to be viewed, instead of solved.

The literature therefore suggests the following path: pure algorithmic methods devoid of any consideration of human factors, local spreadsheet-based methods, and cloud-based integrated systems.

TABLE 2.1: Comparison of Timetabling Methodologies

Feature	Spreadsheets (Legacy)	Pure Algorithms	Modern Cloud Sys- tems
Conflict Detection	Manual & Error- Prone	Fully Automated	Real-time Auto- mated
Data Accessibility	Offline / Localized	Varies (often CLI based)	Cloud-synchronized (Web/Mobile)
Flexibility	High (free-form data entry)	Low (strict mathe- matical constraints)	High (heuristics + constraints)
Integration	None	Low	High (APIs, Database links)
User Interface	Basic Grid	Academic / Non- existent	Role-based Dash- boards

The system designed in this thesis is based on this framework, employing the "Modern Cloud System" framework while incorporating a heuristic parser to overcome the limitations of legacy spreadsheet systems, creating an architectural framework that enables seamless automation of constraint solving systems in the future.

Chapter 3

Technologies and Frameworks

The IIITA Timetabling System utilizes a modern, browser-based stack selected to prioritize high-performance data processing and strict referential integrity. This chapter outlines the technical foundation, focusing on tools that facilitate complex client-side computation and the normalization of institutional data.

TABLE 3.1: Summary of the Technology Stack

Layer	Technology	Primary Function in TTGen
Frontend Framework	React 19	Rendering dense, interactive 5-day time grids.
Programming Language	TypeScript	Enforcing strict data contracts for slots and genes.
Database	PostgreSQL	Maintaining integrity across rooms, faculty, and subjects.
Backend / BaaS	Supabase	API gateway and secure database interfacing.
Excel Parsing	SheetJS (xlsx)	Heuristic extraction of semantic meaning from legacy files.

3.1 Frontend Architecture: React 19 and TypeScript

The user interface is created with the ability to handle dense data interactions without a drop in performance. This is accomplished through the use of React's virtual DOM and TypeScript static type checking.

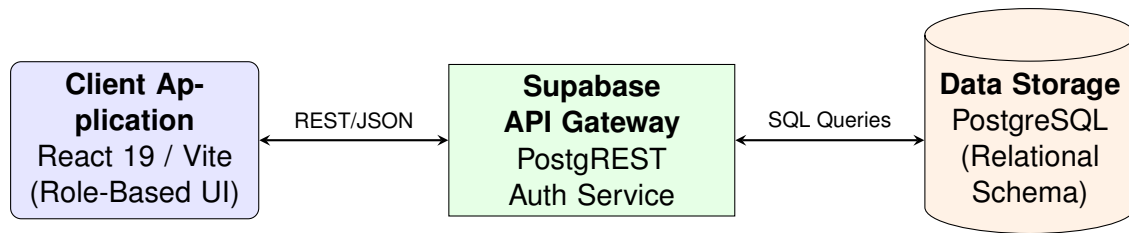


FIGURE 3.1: High-Level System Architecture and Component Interaction

3.1.1 Performance and Data Reliability

For expensive operations like $O(N^2)$ clash detection, memoization of these logic-driven functions is done using the `useMemo` hook, such that these functions are run only if the data from the timetable has been modified. TypeScript is utilized for strict data interfaces, like `ParsedSlot`.

```
// Example TypeScript Interface for Solver Data Contracts
```

```
interface ParsedSlot {  
  
    id: string;  
  
    day: string;  
  
    startTime: string;  
  
    subjectCode: string;  
  
    type: 'Lecture' | 'Tutorial' | 'Practical';  
  
    roomName: string;  
  
    section: string;  
  
}
```

3.2 Construction and Styling: Vite and Tailwind CSS

Both the construction and styling processes are designed to facilitate fast iterations and immediate feedback loops.

- **Vite:** Utilizes native ES Modules, enabling hot module replacement (HMR) almost instantaneously, which is essential during the optimization of grid generation algorithms.
- **Tailwind CSS:** The utility-based CSS framework makes it possible for the application to provide real-time styling feedback, such as a red border around conflicting sessions.

3.3 Back-end and Data Ingestion: Supabase

Static files are replaced with a central PostgreSQL database provided by Supabase with referential integrity enforced using foreign keys.

- **Heuristic Parsing:** The `xlsx` (SheetJS) package is utilized to parse institutional spreadsheets in their raw form. This information is interpreted using an in-house heuristic engine to derive its semantics, such as credit structures in the form of L-T-P and faculty mapping.
- **Authentication:** Authentication for administrators is achieved through Supabase Auth, which is implemented with state management through a global `AuthContext`.

3.4 Solver Engine: Architecture of Pure TypeScript

The most important aspect of the current solution is its pure-TypeScript solver engine, which works directly in the browser. It doesn't require any optimizations on the server side, enabling live generation of schedules. Solver consists of several modules:

- `dataPrep.ts`: Breaks up high-level credits into individual sessions, applying the 2+1 rule of lectures.
- `constraints.ts`: Checks the solutions based on ten hard and three soft constraints.
- `mutations.ts`: Contains five different mutation operators, such as session day changing and Gaussian time shift.
- `solver.ts`: Works as a control center, managing the (1+1)-ES process, elitism, and restarts from stagnation.
- `localSearch.ts`: Applies local search operations to refine the schedule and resolve specific conflicts.

3.5 External Integration: Reporting and Exporting

For workflow efficiency purposes, the system has the following integrations with external reporting systems:

- **Google Sheets API**: Enables exportation of schedules to Google Drive in a frozen header format with session coloring.
- **PDF Output**: Makes use of `html-to-image` and `jsPDF` libraries for snapshot output of the timetable into PDF form.

Chapter 4

System Architecture and Design

This chapter details the structural design of the IIITA Timetabling System. It defines the application topology, the relational database schema, the solver's internal data model, the asynchronous processing design for non-blocking execution, the Excel data ingestion pipeline, and the state-based routing logic used for view management.

4.1 Application Topology

The system architecture consists of three primary components:

- **Frontend (React SPA):** A Single Page Application that handles user interaction, clash detection, and PDF generation. It interacts with the backend through REST API calls.
- **Backend (Supabase):** Provides the API gateway, manages user authentication, and handles database interactions.
- **Database (PostgreSQL):** A relational database that stores institutional data, including users, rooms, subjects, faculty, and the generated timetable slots.

4.2 Database Schema Design

The database schema is normalized to ensure data integrity and minimize redundancy. It consists of eight tables: six core entity tables and two mapping tables for the solver engine. Figure 4.1 illustrates the Entity-Relationship (ER) diagram.

4.2.1 Core Entities

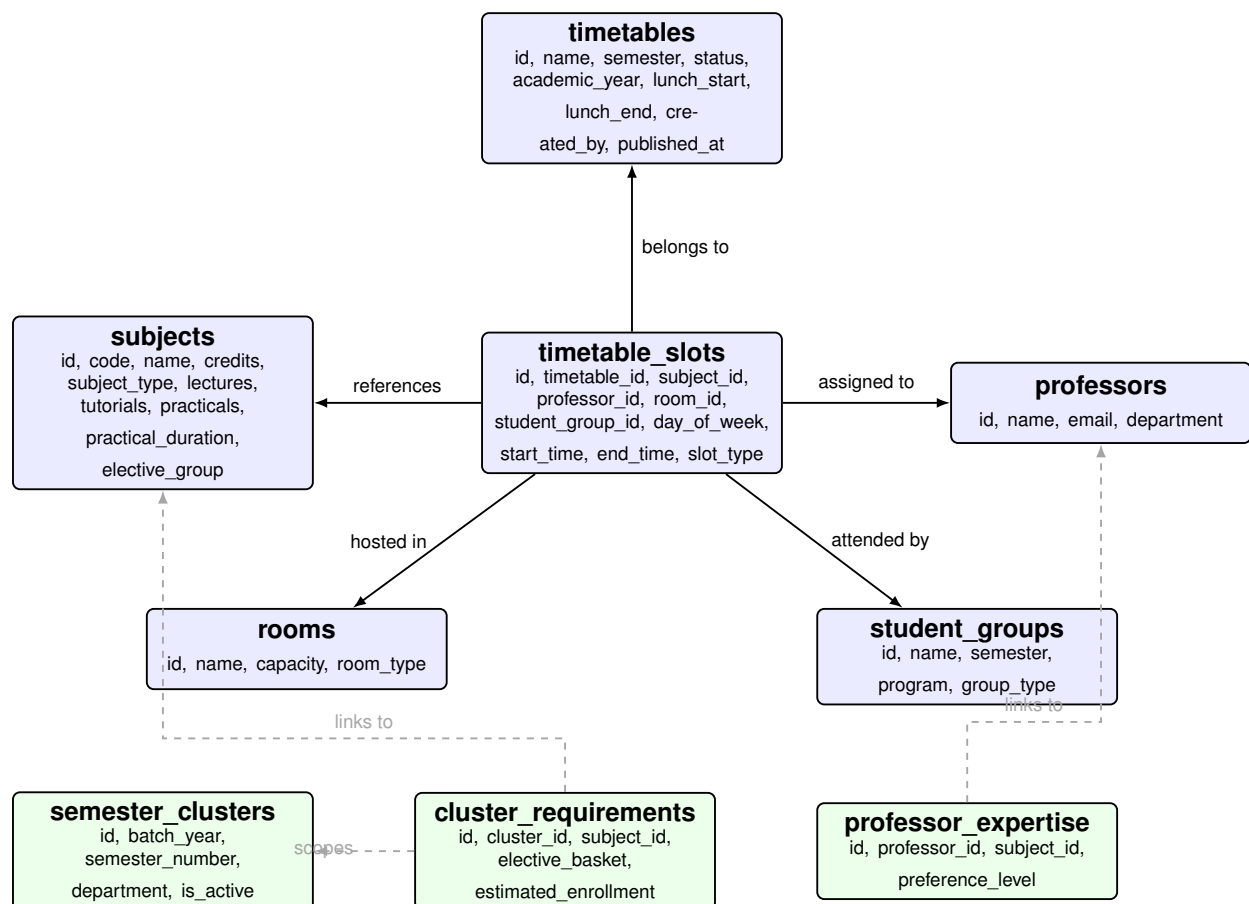


FIGURE 4.1: Entity-Relationship Diagram of the Complete Database Schema

- **timetables**: The container for a specific schedule. It stores the iteration name, semester, academic year, and lunch break constraints.
- **subjects**: Contains course metadata, including the course code, credit structure (L-T-P), and elective grouping.
- **professors**: Stores faculty information, uniquely identified by email.

- **rooms:** Physical room data including capacity and room type (Lecture Hall or Laboratory).
- **student_groups:** Defines student cohorts by program, semester, and section.

4.2.2 The Central Hub: `timetable_slots`

The `timetable_slots` table is the primary junction table connecting timetables to subjects, professors, rooms, and student groups. Table 4.1 details its structure.

TABLE 4.1: Schema Specification for `timetable_slots`

Column Name	Data Type	Description & Constraints
<code>id</code>	UUID	Primary Key, auto-generated.
<code>timetable_id</code>	UUID	Foreign Key referencing <code>timetables</code> .
<code>subject_id</code>	UUID	Foreign Key referencing <code>subjects</code> .
<code>professor_id</code>	UUID	Foreign Key referencing <code>professors</code> .
<code>room_id</code>	UUID	Foreign Key referencing <code>rooms</code> .
<code>student_group_id</code>	UUID	Foreign Key referencing <code>student_groups</code> .
<code>day_of_week</code>	Integer	1 = Monday, ..., 5 = Friday.
<code>start_time</code>	VarChar	24-hour format (e.g., "09:00").
<code>end_time</code>	VarChar	24-hour format (e.g., "11:00").
<code>slot_type</code>	VarChar	Enum: 'Lecture', 'Tutorial', 'Practical'.

In order to facilitate the automated generation process, the schema contains a series of mapping tables that are utilized by the solver engine:

- **semester_clusters:** Clusters together subjects based on batch, semester, and department to determine the scope of the schedule.
- **cluster_requirements:** Establishes mappings of subjects to clusters with electives and estimates of the number of students enrolled.

- **professor_expertise**: Determines the subjects that a professor is able to teach along with their preferences.

4.2.3 Solver Data Structures

The solver engine works off a specific set of TypeScript interfaces which model the scheduling problem into a Genetic Algorithm chromosome. See Figure 4.2 for details of the interfaces involved.

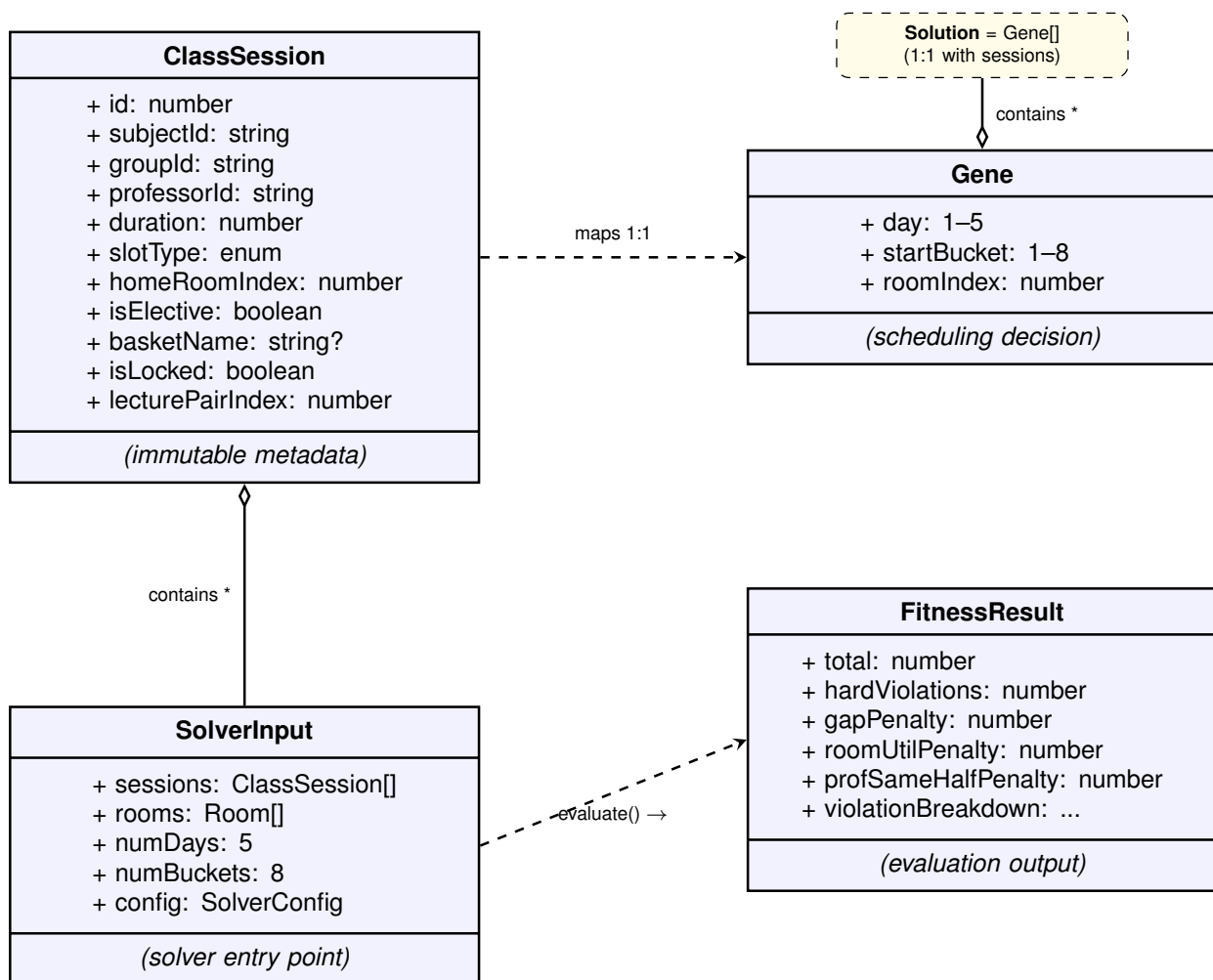


FIGURE 4.2: UML Class Diagram of the Solver Data Model

4.3 Asynchronous Processing Design

To execute the (1+1)-ES solver without blocking the browser's single-threaded UI, the following strategy is used:

- **Single-Threaded Execution:** Since JavaScript runs on a single main thread, a continuous (1+1)-ES loop would block UI updates and result in "unresponsive script" warnings.
- **Asynchronous Batched Execution:** The solver partitions the optimization workload into discrete batches of generations, preventing the CPU from becoming an uninterruptible bottleneck.
- **Event Loop Yielding:** After each batch, the system utilizes `setTimeout(..., 0)` to yield control back to the browser's event loop, allowing for essential tasks like input handling.
- **UI Responsiveness:** This non-blocking design enables the application to render real-time fitness charts and maintain a fluid, interactive user experience throughout the optimization process.

The system includes a data ingestion module (`TimetableImporter`) to parse existing Excel-based timetables into the normalized database schema. The process follows a five-step pipeline:

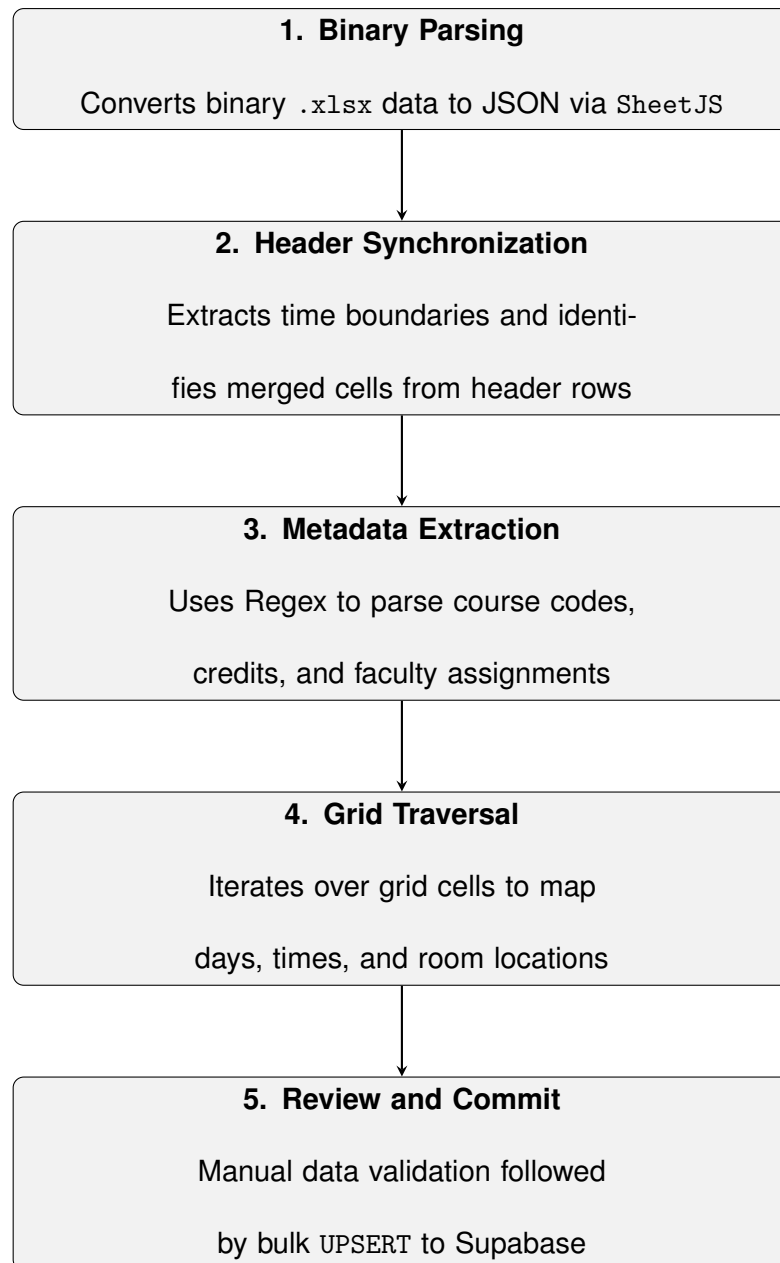


FIGURE 4.3: Data Ingestion Pipeline for Excel Timetables

- 1. Parsing Binary Files:** Uses the 'SheetJS' library to parse '.xlsx' files and convert them to JSON arrays.
- 2. Header Synchronization:** Detects time-slot boundaries from the header row, accommodating different time formats and determining merged cells that contain multi-hour slots.

3. **Extracting Metadata:** Uses Regular Expressions to determine the subjects, instructors, and credit formats. Deals with advanced strings containing multiple sections in one assignment.
4. **Traversing the Grid:** Converts each grid cell to its corresponding day and time slot, along with room number and section data.
5. **Review and Commit:** Offers a user interface for manual editing before executing batch UPSERT actions on Supabase.

There is no URL-based navigation; instead, state-based view management is used. The `currentView` state in the root component controls which module is active.

- `import`: Renders the `TimetableImporter` view.
- `generator`: Renders the `GeneratorView` wizard with multiple steps to generate a timetable automatically.
- `editor`: Renders the `EditorView` where the user can create a timetable using LNS drag-and-drop with automatic fixing.
- `student`: Renders the `TimetableViewer` view (for All Students administrative view).
- `prof`: Renders the `ProfessorTimetableViewer` view.
- `room`: Renders the `RoomTimetableViewer` view.
- `master-room`: Renders the `MasterRoomViewer` view.
- `free-room`: Renders the `FreeRoomViewer` view.

This makes sure that only the code necessary for the active module is loaded. The custom hooks (`useGeneratorData`, `useEditorData`) handle data loading and caching, which keeps the data and user interface layers separate.

Chapter 5

Implementation and Results

5.1 Introduction to the Optimization Engine

The transition from data ingestion to the core solver introduces a highly constrained NP-hard optimization problem. The IIITA Timetabling System models this challenge mathematically, solving it via client-side heuristics. Rather than relying on traditional deterministic approaches which fail to scale, the system employs an evolutionary search mechanism. The engine processes high-dimensional inputs—comprising student cohorts, faculty assignments, and spatial capacities—and systematically navigates a vast, discontinuous search space to identify a globally optimal, conflict-free schedule.

5.2 Mathematical Representation of the Schedule

In order to achieve computation of optimal schedules, the schedule is first transformed to a particular data structure, namely *Chromosome*. In the solver, timetable S is considered to be represented by a sequence of *genes*.

Genes are represented in the following form:

$$g_i = (d_i, t_i, r_i) \tag{5.1}$$

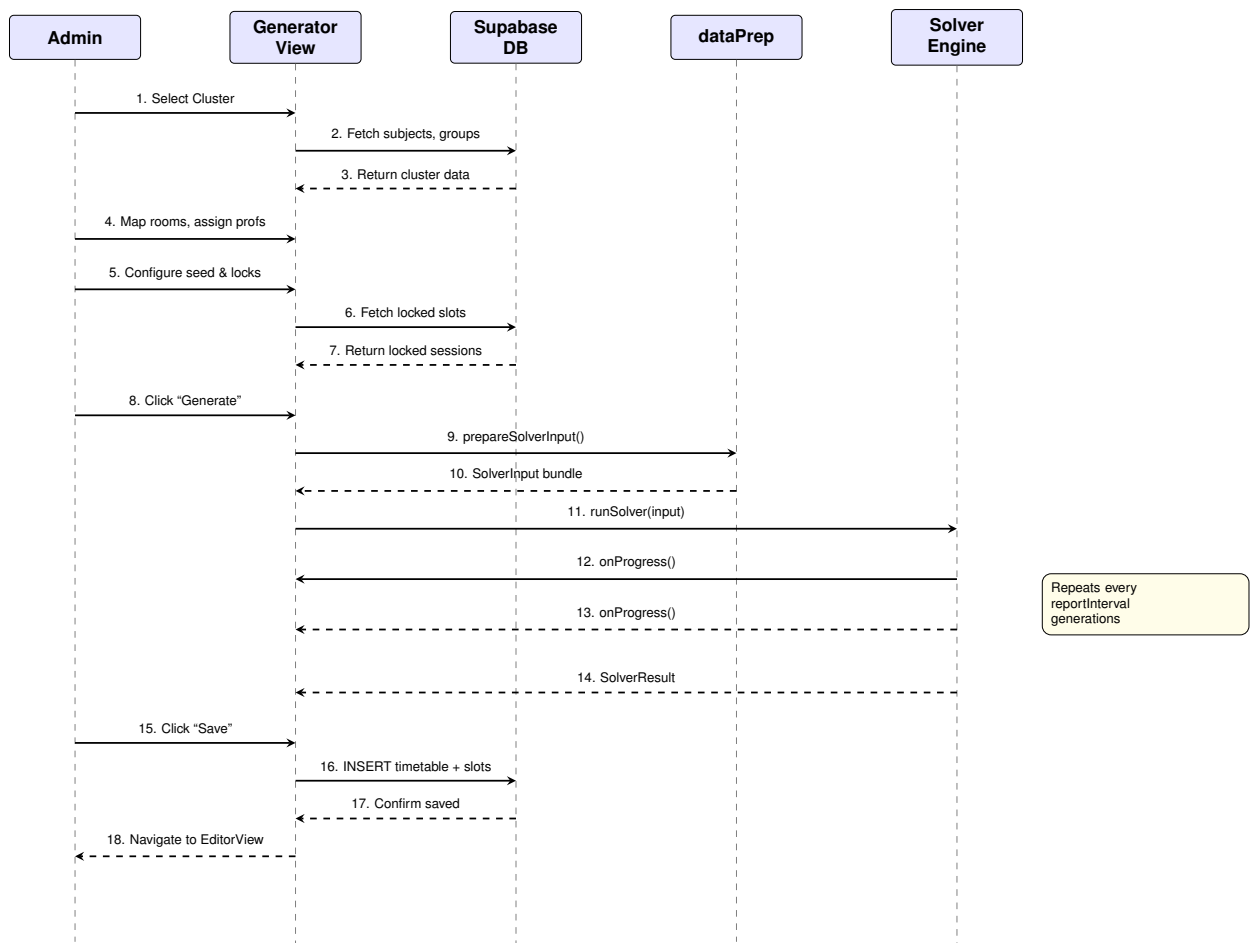


FIGURE 5.1: Sequence Diagram: End-to-End Intelligent Generator Workflow

such that:

- $d_i \in \{1, 2, 3, 4, 5\}$ - the number of the day of the week (Monday-Friday).
- $t_i \in \{1, 2, \dots, 8\}$ - the number of time period (time period represents each hour starting at 08:50 and finishing at 18:30).
- $r_i \in \mathcal{R}$ - the index in the list of physical rooms' numbers.

Each gene is related to the particular and immutable `ClassSession` object containing information about the session itself (the subject name, the students' group G_i , professor p_i , session's duration D_i , and the type of the slot (Lecture, Tutorial, Practical)).

5.3 Formulation of Hard/Soft Constraints

Optimization of the problem depends on the use of a hybrid function to assess the feasibility and the quality of the solution S under consideration.

5.3.1 Hard Constraints (Feasibility Space)

These constraints set the limit for feasible solutions. In case of violation of any of the hard constraints, the proposed schedule cannot be considered feasible. Overlap condition between sessions i and j may be written as follows:

$$\text{overlap}(t_i, t_j) = \begin{cases} 1 & \text{if } \max(t_i, t_j) \leq \min(t_i + D_i - 1, t_j + D_j - 1) \\ 0 & \text{otherwise} \end{cases} \quad (5.2)$$

The following hard constraints are enforced by the algorithm:

- **Professor Overlap Penalty:** A professor p cannot be physically present in two different locations at time t .

$$\sum_{i \neq j} I(p_i = p_j \wedge d_i = d_j \wedge \text{overlap}(t_i, t_j)) = 0 \quad (5.3)$$

- **Student Group Overlap Penalty:** A specific student cohort G cannot attend two distinct subjects simultaneously. (Concurrent sessions from the same synchronized elective basket are mathematically exempted).

$$\sum_{i \neq j} I(G_i = G_j \wedge d_i = d_j \wedge \text{overlap}(t_i, t_j) \wedge \neg \text{SameBasket}(i, j)) = 0 \quad (5.4)$$

- **Room Clash Penalty:** A physical room r can host at most one session at any given time.

$$\sum_{i \neq j} I(r_i = r_j \wedge d_i = d_j \wedge \text{overlap}(t_i, t_j)) = 0 \quad (5.5)$$

- **Lunch Boundary Enforcement:** Sessions cannot cross the designated lunch break (B_{lunch} from 13:00 to 14:30, bounded between slot 4 and 5).

$$\forall i \in S, \quad [t_i, t_i + D_i - 1] \cap B_{\text{lunch}} = \emptyset \quad (5.6)$$

- **L-T-P “Chunking” (The 2+1 Rule):** For core courses with $L \geq 2$ credits, the algorithm mathematically forces the contact hours into a contiguous 2-hour block and subsequent 1-hour blocks, distributed across distinct days. Let $\mathcal{L}_c$ be the set of lecture sessions for subject c :

$$\exists! k \in \mathcal{L}_c : D_k = 2 \quad \text{and} \quad \forall j \in \mathcal{L}_c \setminus \{k\}, D_j = 1 \quad (5.7)$$

$$\forall i, j \in \mathcal{L}_c, i \neq j \implies d_i \neq d_j \quad (5.8)$$

TABLE 5.1: Hard Constraints Enforced by the Solver

#	Constraint Name	Description
1	Time Boundary	The maximum slot for conducting a session is slot 8 (18:30).
2	Break / Lunch Crossing	Multi-slot sessions cannot cross a break boundary ($2 \rightarrow 3, 4 \rightarrow 5$).
3	Room Non-overlap	Only one session can be conducted in a room at any (day, slot) pair.
4	Professor Non-overlap	A professor can conduct only one session at any (day, slot) pair. There is no elective exemption.
5	Group Non-overlap	Only one session can be attended by the same student group at any (day, slot) pair. Elective - elective combinations and basket combinations are exempt from this rule.
6	Elective Baskets Synchronization	The synchronized baskets must have the same day and slot. Two different baskets cannot have the same slot (global anti-clash).
7	Lab Room	Practical Sessions must be scheduled in the rooms whose types are "Lab".
8	WMC-Section Clash	The whole-batch (WMC) session cannot coincide with any other session in the same slot.
9	Home Room	The core lectures and tutorials must be scheduled in their respective home rooms.
10	2 + 1 Lecture Rule	Double lecture sessions for the same course and student group should not take place on the same day as the single lectures.

5.3.2 Soft Constraints (The Quality Space)

Once the solution space is feasible, the engine optimizes for schedule quality by minimizing soft penalties.

- **Minimizing Student Gaps:** The engine minimizes idle slots between a cohort's first and last class on any given day. Let $\mathcal{T}_{g,d}$ be the set of all active time slots for student

group g on day d . The gap penalty $G(S)$ is:

$$G(S) = \sum_g \sum_d ((\max(\mathcal{T}_{g,d}) - \min(\mathcal{T}_{g,d}) + 1) - |\mathcal{T}_{g,d}|) \quad (5.9)$$

- **Room Capacity Optimization:** The system matches class enrollment c_i to room capacity K_{r_i} . Let the utilization ratio be $u_i = c_i/K_{r_i}$. Given optimal thresholds $\alpha = 0.6$ and $\beta = 1.2$, the room utilization penalty $R(S)$ is formulated as:

$$R(S) = \sum_i (\max(0, \alpha - u_i) + \max(0, u_i - \beta)) \quad (5.10)$$

5.4 The (1+1) Evolution Strategy Solver

The optimization engine is powered by a (1+1) Evolution Strategy (ES) algorithm. It relies on iterative stochastic perturbations and adaptive mutation rates to navigate the constrained schedule space.

5.4.1 Strict Elitism

The algorithm employs a strictly elitist selection mechanism. In each generation, a parent solution S_{parent} undergoes mutation to produce a single offspring S_{child} . The offspring replaces the parent if and only if its total fitness score is less than or equal to the parent's score:

$$S_{parent}^{(t+1)} = \begin{cases} S_{child} & \text{if } f(S_{child}) \leq f(S_{parent}^{(t)}) \\ S_{parent}^{(t)} & \text{otherwise} \end{cases} \quad (5.11)$$

This greediness guarantees that the solver never loses ground on hard constraint feasibility.

5.4.2 Mutation Operators

To generate S_{child} , the engine selects a subset of mutable sessions and applies highly stochastic operators:

- **Gaussian Time Shifts:** The start time t_i is perturbed using Gaussian noise scaled by variance σ :

$$t_i^{(new)} = t_i + [\mathcal{N}(0, \sigma^2)] \quad (5.12)$$

- **Type-Aware Room Reassignments:** A session is moved to a randomly selected room $r_k \in \mathcal{R}$ that strictly matches its required architectural type (e.g., Lab vs. Lecture Hall).
- **Targeted Swaps:** Two sessions i and j of identical duration exchange their spatial-temporal coordinates, effectively jumping out of local minima without increasing the overall density.

5.4.3 Adaptive Variance (The Brain)

To dynamically balance global exploration and local exploitation, the solver implements Rechenberg's 1/5th Success Rule. The mutation strength σ is adapted every $W = 50$ generations based on the success rate p_s (the ratio of offspring that successfully improved or matched fitness).

$$\sigma \leftarrow \begin{cases} \sigma \times 1.22 & \text{if } p_s > 0.20 \quad (\text{Search wider}) \\ \sigma \times 0.82 & \text{if } p_s < 0.20 \quad (\text{Search finer}) \\ \sigma & \text{otherwise} \end{cases} \quad (5.13)$$

5.4.4 Stagnation Recovery

In highly dense optimization landscapes, the algorithm may converge to a deep local optimum (a plateau). The engine mathematically tracks this stagnation. If S_{parent} fails to improve for 15,000 consecutive generations, a stagnation recovery protocol is triggered. The current candidate is discarded, and maximum entropy is injected by generating a completely new randomized initial timetable.

5.5 Interactive Editor & Large Neighbourhood Search (LNS)

The user interface functions primarily as an engineering tool designed for granular control over the algorithmic output. When manual drag-and-drop actions introduce systemic conflicts, an integrated Large Neighbourhood Search (LNS) is deployed to repair the grid.

The “Auto-Fix” feature applies a destroy-and-repair heuristic. Instead of recalculating the entire grid, it isolates the localized clashing subset $C \subseteq S$. The LNS algorithm iterates up to 1,500 times, randomly selecting a session $i \in C$, applying stochastic mutations (relocation, time shifts, or swaps), and evaluating the local differential fitness. If a localized mutation resolves the clash without violating surrounding sessions, the repair is committed.

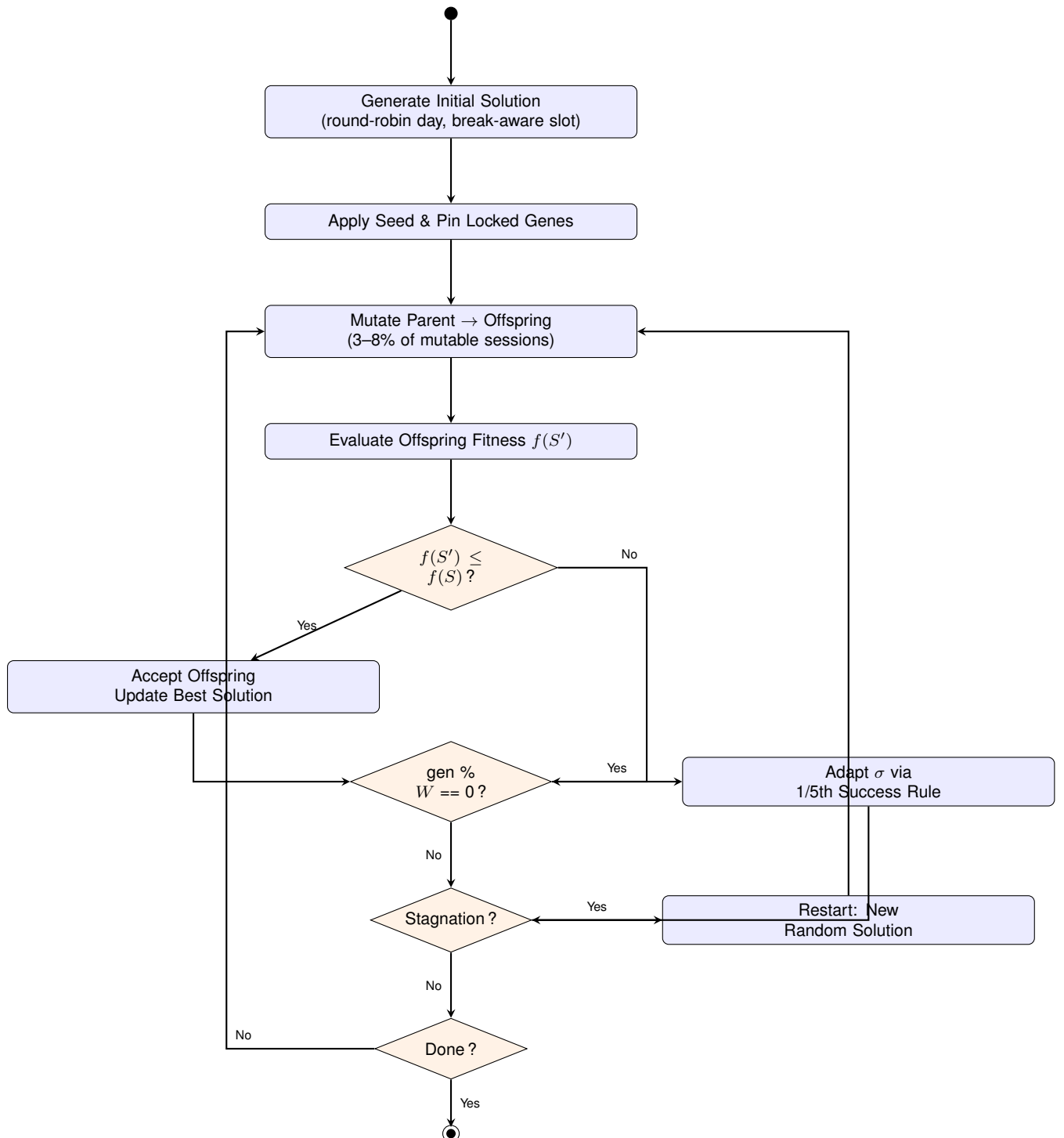


FIGURE 5.2: Activity Diagram: (1+1)-ES Solver Execution Flow

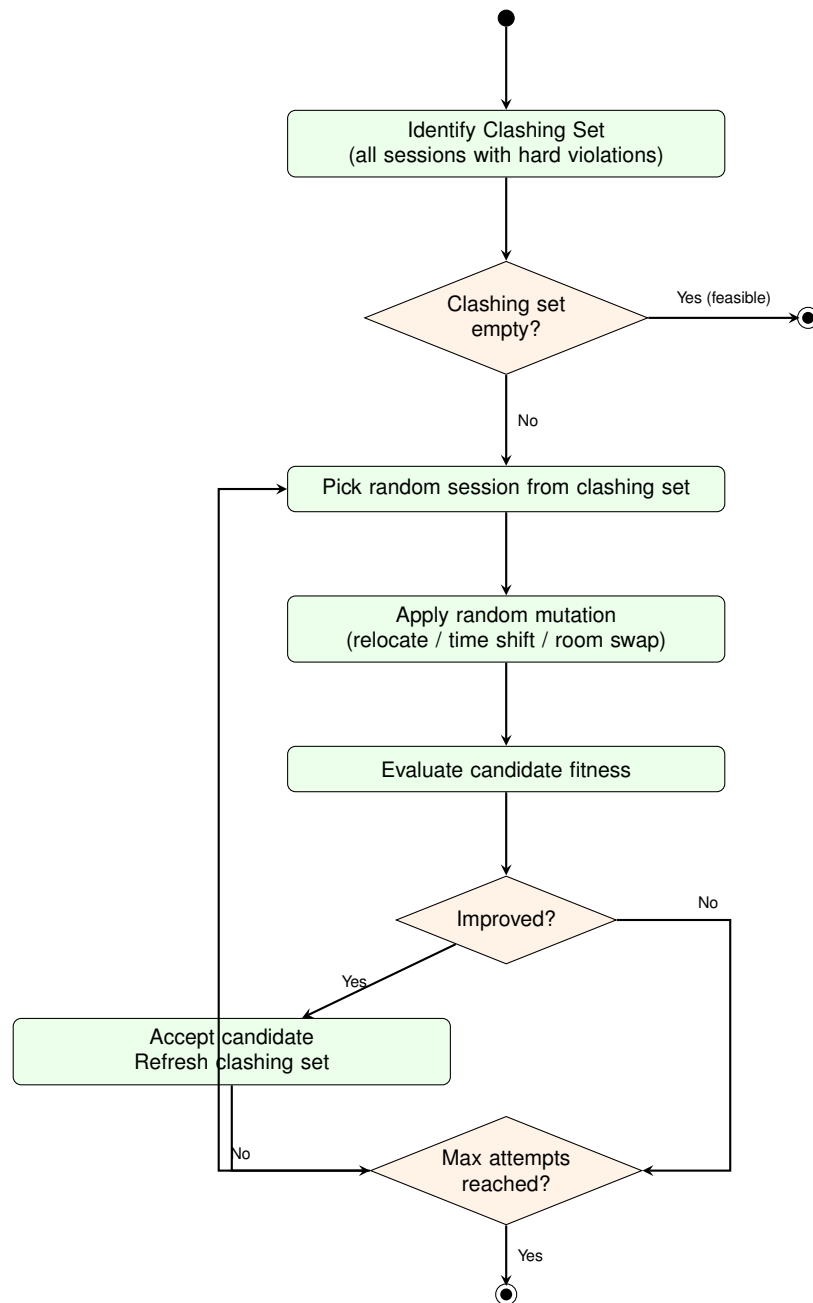


FIGURE 5.3: Activity Diagram: LNS Destroy-and-Repair Auto-Fix

5.6 Results and Application Features

The implementation of the optimization engine drastically reduces the administrative overhead. By executing entirely client-side, the system achieves feasibility rapidly, circumventing backend server latencies.

5.6.1 Algorithmic Convergence

The solver demonstrates rapid convergence. The heavily weighted hard penalties force the evolution strategy to prioritize feasibility before aggressively optimizing the soft quality metrics.

‘[Insert Image: Fitness Score vs. Generations Graph]’

5.6.2 The Interactive Drag-and-Drop Grid

The primary workspace visualizes the temporal matrix. It calculates multi-dimensional clashes in real-time, surfacing spatial and temporal overlaps through distinct programmatic warnings.

‘[Insert Image: Main UI Grid showing Clashes]’

5.6.3 LNS Auto-Fix Execution

When administrators execute the LNS repair routine, the interface explicitly highlights the target neighbourhood and dynamically resolves overlapping nodes without destabilizing the optimized framework.

‘[Insert Image: UI highlighting Auto-Fix suggestions]’

5.6.4 Export and PDF Generation

The final scheduling matrix is programmatically serialized into a high-fidelity PDF, leveraging DOM capture logic to bypass CSS layout constraints, ensuring the complete multi-dimensional array is rendered cleanly.

‘[Insert Image: Final PDF Timetable Output]’

5.7 Dynamic Time Grid Generation

One of the major problems with rendering timetables is the flexibility of time blocks. Defining a fixed grid with a predetermined number of columns, such as ten 1-hour slots between 9:00 AM and 7:00 PM, leads to underutilization of the screen space and inability to represent unusual time blocks, e.g., a 1.5-hour lesson.

This problem is solved using a **Dynamic Time Column Generator** algorithm, which is implemented by all viewers in the system. When a user selects a timetable, the system extracts all corresponding `timetable_slots`, and the following actions are taken:

1. All unique `start_time` and `end_time` values are extracted from the dataset.
2. Hardcoded institutional breaks are inserted into the list: 10:50 – 11:00 (morning break) and 13:00 – 14:30 (lunch break).
3. All time points are sorted in chronological order to produce a list of discrete time intervals.
4. Pairs of adjacent time intervals are used to produce a series of standard columns.
5. All columns that lie inside the lunch period or breaks and have no lessons assigned are filtered out.

As a consequence, a very dynamic grid is produced, with only active time blocks displayed on the screen.

5.8 Multi-Dimensional Clash Detection Algorithm

One of the most critical administrative features is the automated detection of scheduling conflicts. The 'TimetableViewer' component implements a robust $O(N^2)$ algorithm that compares every slot against every other slot within the selected timetable to detect overlaps.

An overlap is mathematically defined as occurring when:

$$(SlotA.start < SlotB.end) \wedge (SlotA.end > SlotB.start) \quad (5.14)$$

When an overlap is detected, the algorithm categorizes the clash into one of four distinct types as summarized in Table 5.2.

TABLE 5.2: Types of Scheduling Clashes Detected

Clash Type	Logical Condition and Description
Room Clash	$SlotA.room == SlotB.room$ Two different classes are assigned to the exact same physical space simultaneously.
Professor Clash	$SlotA.professor == SlotB.professor$ A single faculty member is booked to teach two different classes at the same time.
Section Clash	$SlotA.section == SlotB.section \wedge SlotA.subject \neq SlotB.subject$ A specific student cohort is expected to attend two distinct subjects simultaneously.
WMC-Section	Specialized institutional rule where generic weekend/WMC sessions overlap with standard section sessions.

These clashes are immediately surfaced to the administrator in a dedicated visual panel, complete with distinct iconography (e.g., a Map Pin for Room clashes, a User icon for Professor clashes), allowing for rapid identification and remediation.

5.9 Inverted Logic: The Free Room Finder

Traditional scheduling systems focus on when rooms are occupied. However, students and staff frequently need to know when rooms are *available*. The 'FreeRoomViewer' component achieves this through an inverted logic algorithm.

Unlike the dynamic grid, the Free Room Finder establishes a hardcoded temporal matrix (e.g., 8:50 AM to 6:30 PM in discrete blocks).

The algorithm executes as follows:

1. The system first queries the database for **all** existing rooms.
2. It then queries the `timetable_slots` to retrieve all currently active sessions.
3. For every cell in the temporal matrix (Day $\times$ Time), it instantiates the complete set of all campus rooms.
4. It iterates through the active sessions. If a session falls within the current cell's Day and Time, that session's `room_id` is removed from the set of available rooms.

The remaining rooms in the set are then rendered in the UI as green badges, instantly showing users where they can find empty spaces across the campus.

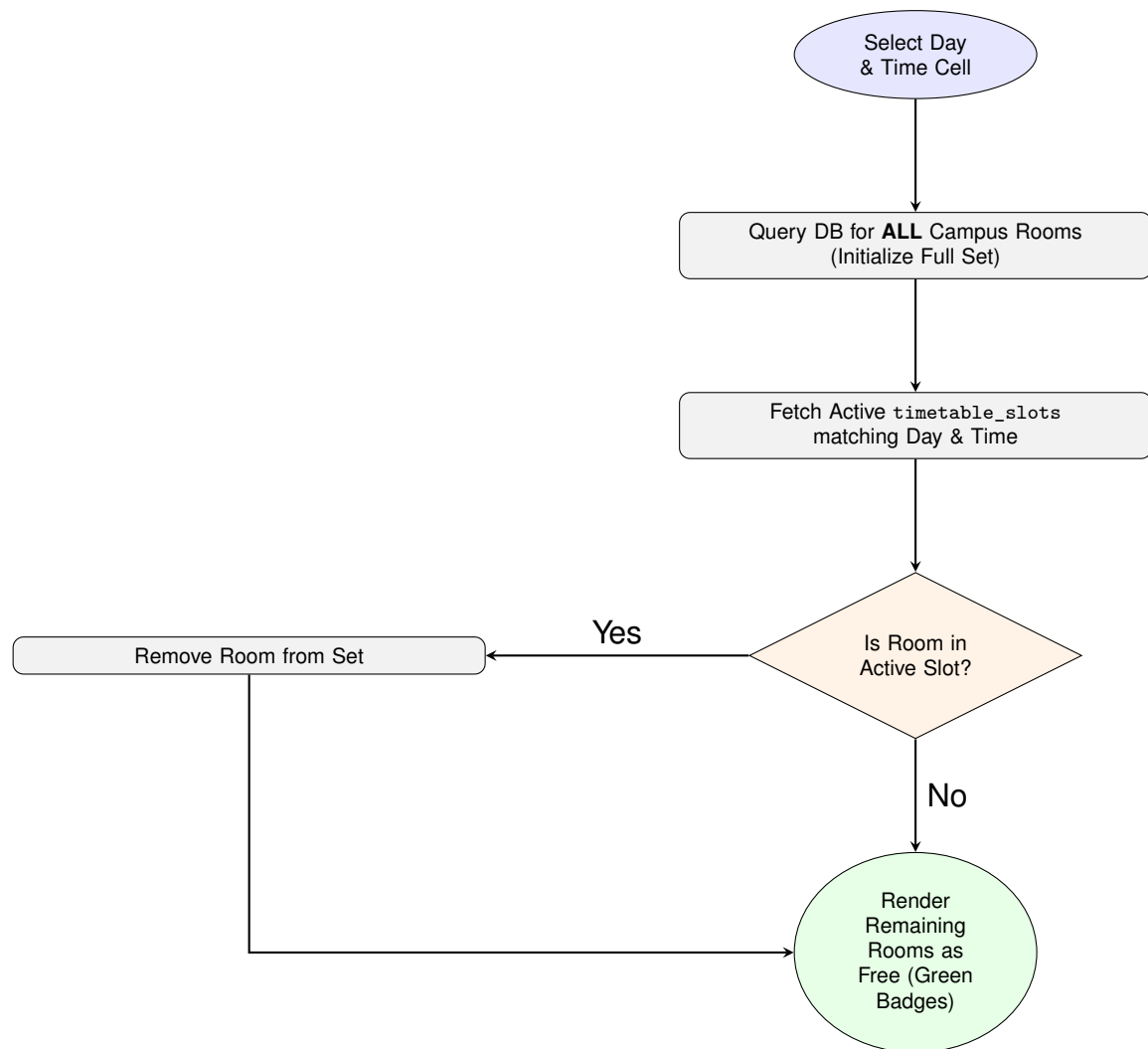


FIGURE 5.4: Inverted Logic Flowchart for the Free Room Finder Algorithm

5.10 React Component Hierarchy

To maintain a scalable and manageable codebase, the frontend relies on a strict component tree hierarchy. Figure 5.5 illustrates the relationship between the global layout and the localized, role-based timetable viewers.

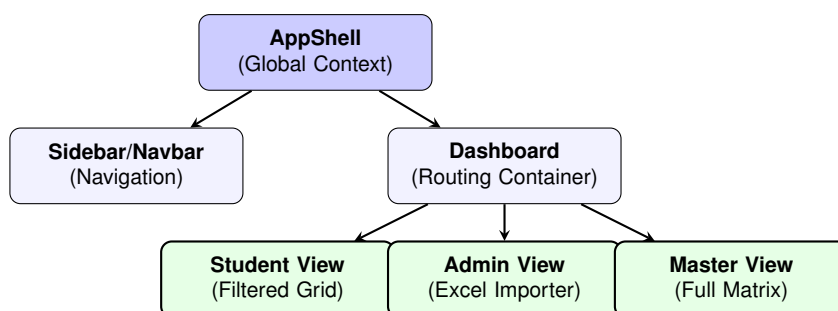


FIGURE 5.5: Hierarchical Structure of the Core React Components

5.11 Role-Based Viewer Implementations

The system provides several specialized views tailored to different stakeholders, summarized in Table 5.3.

TABLE 5.3: Role-Based Component Mapping and Data Scope

Stakeholder Role	Component View	Primary Filter Parameter
Student	TimetableViewer	Semester & Specific Section
Professor	ProfessorTimetableViewer	Faculty Email/ID
Department Admin	RoomTimetableViewer	Specific Room Name
Campus Admin	MasterRoomViewer	Building prefix (e.g., “CC-3”, “LT”)
Facility Staff	FreeRoomViewer	Temporal Matrix (Empty Rooms)

5.11.1 Student Cohort View

Given the complexity of modern curricula featuring numerous electives and minor tracks, a master timetable can be overwhelming. The `TimetableViewer` allows users to filter the

master schedule by selecting a specific semester and section.

Crucially, it implements a **Smart Merging algorithm**. Often, core lectures are held for "All Sections" simultaneously, while practical laboratory sessions are split by specific section (e.g., "Sec A"). The merging algorithm intelligently filters the master data, pulling in slots explicitly assigned to the selected section *and* slots assigned to the generic "All" section (WMC), combining them into a single, cohesive cohort schedule.

5.11.2 Master Occupancy Viewer

For campus administrators, the 'MasterRoomViewer' provides a macro-level perspective. It displays a dense grid where rows represent physical rooms rather than student groups. Because an institution may have hundreds of rooms, this view implements a building-level filter. The system extracts the building prefix from the room name (e.g., identifying "CC-3" from "CC-3-5106") and allows the administrator to filter the grid, rendering a manageable overview of specific campus zones.

5.12 Export Approaches and Outcomes

To fill in the gap between the digital world and reality, there is an effective set of export options implemented by the tool.

5.12.1 Client-Side PDF Export

It is possible to generate a PDF of the current screen view. This function uses 'html-to-image' package that generates a snapshot of the current DOM state as a PNG image and

embeds it to the 'jsPDF' document. One of the key problems solved during implementation is related to CSS sticky property that needed to be temporarily turned off right before capturing the screen for accurate image generation.

5.12.2 Google Sheets Export

Exporting to Google Sheets is supported both in 'RoomTimetableViewer' and 'Free-RoomViewer'. Using the services provided by Google Identity Service client, the React app creates batch requests that both save the raw string data into cells of the Google Sheet, as well as perform additional operations, like formatting the sheet (creation of sheets corresponding to particular rooms, freeze of header rows, assigning pink/gray background color to them and adding solid borders).

5.12.3 User Interface Evaluation

The end result is a fast, modern interface. By implementing Tailwind CSS, it becomes possible to keep the minimalistic look, yet use distinctive color palette (e.g. green for tutorial time and orange for practical classes). Absence of page reloading in between viewing and instant clash detection marks the revolutionary shift from spreadsheet processing to our solution.

5.13 The Interactive Timetable Editor

The `EditorView` component is where human discretion comes into play with the output of the algorithm. This component presents an interactive drag-and-drop interface to refine timetables—both solver-generated and manually imported timetables in Excel format.

5.13.1 Drag-and-Drop Session Rescheduling

The editor provides a dense grid of 5 days $\times$ N slots, where each allocated session is represented by a draggable card. The administrator can drag each session card to a different cell to adjust the session's day and start time. The system calculates the end time of the session and ensures that the newly allocated position is not during the lunch or break hours. Each drag operation pushes the old state onto a stack of undo levels, which is limited to 30 items.

5.13.2 Inline Editing

Each session card presents an inline editing tool when hovered over, which enables the administrator to change the following details:

- **Room Allocation:** A cascading selection interface where the administrator first chooses between Lecture and Lab rooms and then selects from the filtered room list.
- **Professor Allocation:** A drop-down list containing all the professors in the university.

5.13.3 Real-Time Conflict Visualization

The editor incorporates a client-side conflict checking algorithm which is called with every state update. Sessions in a conflicting set receive a distinctive red background and ring border. The algorithm performs the exact same validations as the solver, which includes: room overlaps, professor overlaps, section overlaps (except electives), WMC-section conflicts, basket elective conflicts, and the 2+1 same day lecture restriction.

5.13.4 Feasibility Check

The "Check Feasibility" button builds a comprehensive `SolverInput` using the current state of the editor, translates the slot positions to a `Solution` (Gene array), adds the locked sessions of all other timetables, and calls the solver's `evaluate()` method. The result shows the fitness value and violation count per constraint, which provides the administrator a detailed picture of unresolved problems.

5.13.5 LNS Auto-Fix

If there are any conflicts, the administrator can call the **Large Neighborhood Search (LNS) Auto-Fix** [shaw1998using], which:

1. Enumerates all indices corresponding to a session where there is any violation in the hard constraints ("clashing set").
2. Runs until at most 1500 attempts have been made. During each attempt, one randomly chosen session from the clashing set is randomly mutated (either relocate, reschedule, or room swap).
3. Should a mutation result in a higher total fitness value, the new solution is accepted and the clashing set is regenerated.
4. Prematurely ends should all hard constraint violations be nullified.

Because this LNS algorithm only considers the sessions involved in constraint violations, instead of optimizing all of the sessions as is done by running a complete solver, the LNS procedure is much faster but still retains the administrator's customizations to non-clashing sessions.

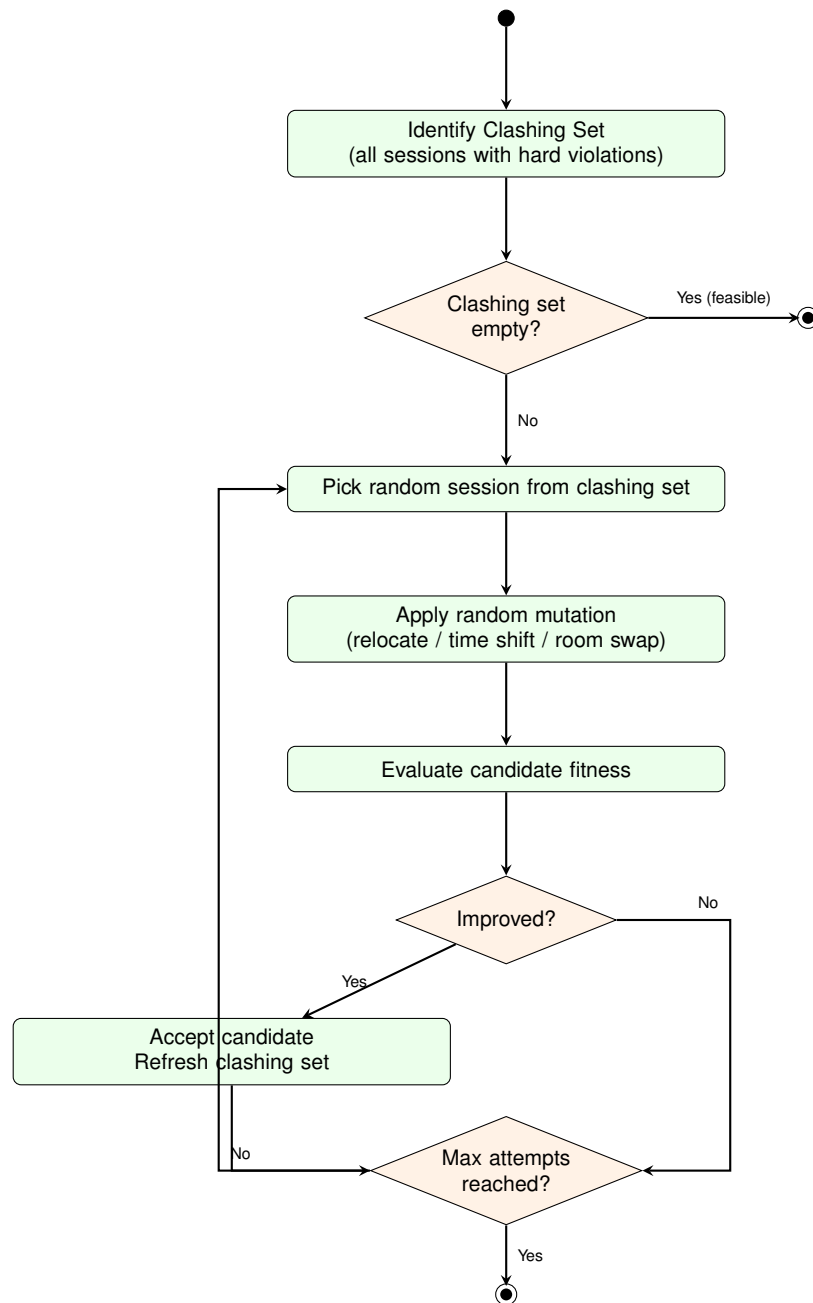


FIGURE 5.6: Activity Diagram: LNS Destroy-and-Repair Auto-Fix

Chapter 6

Conclusion and Future Work

This project replaced the manual, spreadsheet-driven scheduling process at IIIT Allahabad with a web-based platform for the automated generation and management of university timetables. The system provides a centralized environment for data ingestion, conflict-aware editing, and automated scheduling using evolutionary algorithms.

6.1 Summary of Contributions

Key outcomes of the project include the development of a working scheduling system featuring the following:

- **Heuristic Ingestion Engine:** A parser for Excel sheets using `SheetJS` to normalize unstructured data into the PostgreSQL database containing subject information, faculty allocations, and credits.
- **Conflict Detection Algorithm:** A multi-dimensional $O(N^2)$ algorithm which detects conflicts based on rooms, faculties, and students' classes, as well as whole-batch (WMC) constraints.
- **Evolution Strategy Solver:** A (1+1)-ES implementation that evaluates schedules against ten hard and three soft constraints. It utilizes Rechenberg's 1/5th success

rule for adaptive mutation control and a stagnation restart mechanism.

- **Interactive Editor:** An interactive editor that employs drag and drop along with the React framework, including the visualization of conflicting sessions and LNS for automatic fixing of problems.
- **Multistep Configuration Process:** A multistep configuration process where users specify semester groupings, assign home rooms, allocate teachers, and run the solver afterward.
- **Independence of Sessions Across Semesters:** Blocking of sessions from earlier schedule releases to prevent any clashing classrooms or lecturers across courses.
- **Architecture:** The system will be built using a scalable architecture involving React 19, TypeScript, and Vite on the frontend, and Supabase for authentications and database storage of related data.
- **Exports:** Support for client-side PDF export and Google Sheets exports for administrative purposes.

6.2 Future Work – Phase 2 – Integration with M.Tech

The first step in future development will be the integration of M.Tech courses within the automatic solver. The complexity of the algorithm will increase because of a paradigm shift from one-tier scheduling to multi-tier scheduling having interdependencies amongst itself, i.e., the “Basket Tethering” Problem.

6.2.1 The Basket Tethering Problem

The three specializations of M.Tech courses (MLIS, HCI, and CLIS) do not operate independently of each other in the respect that they tend to select courses in certain “baskets” offered under B.Tech VIth sem. The consequence of such selection is that the M.Tech schedules are dependent on those of the B.Tech, such that M.Tech can only be scheduled after specific time-slots have been allocated. The future plan will be to adopt a hierarchical solving approach by assigning anchors as common windows amongst the two schedules.

6.2.2 Solving Core Duality

There are many instances where there exist core duality in courses as some M.Techs consider a subject as core for themselves while for others it is elective (for example: Deep Learning and NLP). The next task is to schedule the dual course once.

6.2.3 Meta-Grouping and Collision Handling

In order to handle the added complications of the scenario, the following meta grouping method is suggested.

- The specializations would be grouped into disjointed meta-groups (e.g., Meta-Batch $A = \{\text{MLIS, RMI, HCI}\}$) based on the common subject area they share.
- A sequence of steps for addressing the issue of the “Three-Way Collision” should be determined.

6.2.4 Scalability of Algorithmic Solution

Given the growing number of constraints because of the inclusion of M.Tech subjects, it is possible that the $(1 + 1)$ -ES algorithm would need to be reworked with parallel computing using Web Workers or by merging ES and CP.

6.2.5 Technical Requirements at the Core

- **Requirement:** Integration of M.Tech specialization courses within the same framework of B.Tech course structure without any conflict.
- **Algorithms:** Give preference to Hierarchy solution: Anchors $\rightarrow$ M.Tech Core $\rightarrow$ B.Tech Core $\rightarrow$ Soft Constraints.
- **Conflict Handling:** Solve the “Three-Way Clash” problem, avoiding any schedule conflict for instructors working at different levels.

6.3 Conclusion

Timetabling System for IIITA shows that complex issues of timetabling in university settings can be resolved through the use of web technology. The timetables produced by the $(1+1)$ -ES algorithm are consistent with the provided constraints, whereas any manual modification can be performed within the editor application. A serverless approach to processing data with the help of the TypeScript version of the solver is appropriate for educational organizations.

References

- [1] Burke, E., Elliman, D. and Weare, R. A Genetic Algorithm Based University Timetabling System.
- [2] Srivastava, G. and Singh, A. Boosting an evolution strategy with a preprocessing step: application to group scheduling problem in directional sensor networks.
- [3] Srivastava, G., Venkatesh, P. and Singh, A. An evolution strategy based approach for cover scheduling problem in wireless sensor networks.
- [4] Gülcü, A. and Akkan, C. Bi-Criteria Simulated Annealing Algorithms for the Robust University Course Timetabling Problem.

Appendix A

Core Algorithm Snippets

This appendix provides crucial code excerpts from the React 19 codebase, demonstrating the practical implementation of the theoretical concepts discussed in the main chapters.

A.1 Dynamic Grid Generation Logic

The following TypeScript snippet from the `TimetableViewer` component demonstrates how the system extracts unique time boundaries to dynamically generate responsive column headers, preventing the display of empty, unused time blocks.

LISTING A.1: Dynamic Time Column Extraction Algorithm

```
// Extract all unique start and end times from the active slots
const allTimes = useMemo(() => {
  const times = new Set<string>();

  // Inject hardcoded institutional break times
  times.add("10:50");
  times.add("11:00");
  times.add("13:00");
  times.add("14:30");

  // Extract times from all valid timetable slots
  slots.forEach(slot => {
    if (slot.start_time && slot.end_time) {
      times.add(slot.start_time.substring(0, 5));
      times.add(slot.end_time.substring(0, 5));
    }
  });

  // Sort chronologically and format as standard columns
  return Array.from(times).sort().reduce((acc, curr, index, arr) => {
    if (index < arr.length - 1) {
      acc.push(`${curr}-${arr[index + 1]}`);
    }
    return acc;
  }, [] as string[]);
}, [slots]);
```

A.2 Real-Time Clash Detection Filter

This snippet illustrates the $O(N^2)$ comparison logic used to immediately identify Room, Professor, and Section overlaps within the currently selected semester block.

LISTING A.2: Overlap Detection and Categorization Logic

```
// O(N^2) comparison to find all overlapping events
const newClashes = filteredSlots.filter((slot, i) => {
  return filteredSlots.some((otherSlot, j) => {
    // Skip self-comparison
    if (i === j) return false;
    // Must be on the same day to clash
    if (slot.day_of_week !== otherSlot.day_of_week) return false;

    // Check for temporal intersection
    const isTimeOverlap = slot.start_time < otherSlot.end_time &&
      slot.end_time > otherSlot.start_time;

    if (isTimeOverlap) {
      // 1. Room Clash
      if (slot.rooms?.name === otherSlot.rooms?.name) return true;
      // 2. Professor Clash
      if (slot.professors?.name === otherSlot.professors?.name) return
        true;
      // 3. Section Clash (Same section, different subject)
      if (slot.student_groups?.name === otherSlot.student_groups?.name &&
        slot.subjects?.id !== otherSlot.subjects?.id) {
        return true;
      }
    }
  });
});
```

A.3 Solver Fitness Evaluation Function

The following excerpt from `constraints.ts` demonstrates the combined fitness evaluation that scores a candidate timetable against all ten hard constraints and three soft constraints.

LISTING A.3: Combined Fitness Evaluation (`constraints.ts`)

```
export function evaluate(input: SolverInput, solution: Solution):
  FitnessResult {
  const { sessions, rooms, numDays, config } = input;

  // Count all hard constraint violations
  const timeBoundary = countTimeBoundaryViolations(sessions, solution);
  const breakCrossing = countBreakViolations(sessions, solution);
  const roomOverlap = countRoomOverlaps(sessions, solution, rooms);
  const professorOverlap = countProfessorOverlaps(sessions, solution);
  const groupOverlap = countGroupOverlaps(sessions, solution);
  const electiveSync = countElectiveSyncViolations(sessions, solution);
  const labRoom = countLabRoomViolations(sessions, solution, rooms);
  const wmcSectionOverlap = countWMCSectionOverlaps(sessions, solution);
  const homeRoom = countHomeRoomViolations(sessions, solution);
  const twoOneLecture = countTwoOneLectureViolations(sessions, solution);

  ;

  const hardViolations = timeBoundary + breakCrossing + roomOverlap +
    professorOverlap + groupOverlap + electiveSync + labRoom +
    wmcSectionOverlap + homeRoom + twoOneLecture;
```

```

// Compute soft constraint penalties
const gapPenalty = computeGapPenalty(sessions, solution, numDays);
const roomUtilizationPenalty = computeRoomUtilizationPenalty(
  sessions, solution, rooms,
  config.roomUtilizationThreshold,
  config.roomOverutilizationThreshold,
);
const professorSameHalfPenalty = computeProfessorSameHalfPenalty(
  sessions, solution, numDays
);

// Weighted composite fitness
const total = hardViolations * config.hardPenalty
  + gapPenalty * config.gapWeight
  + roomUtilizationPenalty * config.roomUtilizationWeight
  + professorSameHalfPenalty * config.professorSameHalfWeight;

return { total, hardViolations, gapPenalty,
  roomUtilizationPenalty, professorSameHalfPenalty,
  violationBreakdown: { timeBoundary, breakCrossing,
    roomOverlap, professorOverlap, groupOverlap,
    electiveSync, labRoom, wmcSectionOverlap,
    homeRoom, twoOneLecture },
};
}

```

A.4 (1+1)-ES Solver Main Loop

The core optimization loop from `solver.ts`, showing elitist selection, σ -adaptation via the 1/5th success rule, and stagnation restart.

LISTING A.4: (1+1)-ES Main Loop (`solver.ts`)

```

export async function runSolver(
  input: SolverInput,
  onProgress?: (p: SolverProgress) => void,
  cancelToken?: { cancelled: boolean },
  seedSolution?: Solution,
  lockedGenes?: Gene[],
): Promise<SolverResult> {
  const { config } = input;
  let parent = seedSolution
    ? seedSolution.map(g => ({ ...g }))
    : generateInitialSolution(input);

  // Pin locked genes from published timetables
  if (lockedGenes) {
    const offset = input.sessions.length - lockedGenes.length;
    for (let i = 0; i < lockedGenes.length; i++) {
      parent[offset + i] = { ...lockedGenes[i] };
    }
  }

  let bestFitness = evaluate(input, parent);
  let bestSolution = parent.map(g => ({ ...g }));
  let sigma = config.initialSigma;

```

```

let successes = 0;

for (let gen = 1; gen <= config.maxGenerations; gen++) {
  if (cancelToken?.cancelled) break;

  const offspring = mutate(parent, input, sigma);
  // Re-pin locked genes (mutations must not move them)
  if (lockedGenes) { /* ... pin logic ... */ }
  const offFitness = evaluate(input, offspring);

  // Elitist selection: accept if offspring is <= parent
  if (offFitness.total <= bestFitness.total) {
    parent = offspring;
    bestFitness = offFitness;
    bestSolution = offspring.map(g => ({ ...g }));
    successes++;
  }

  // 1/5th rule sigma adaptation every W generations
  if (gen % config.adaptationWindow === 0) {
    const rate = successes / config.adaptationWindow;
    if (rate > 0.2) sigma *= config.sigmaIncrease;
    else if (rate < 0.2) sigma *= config.sigmaDecrease;
    successes = 0;
  }

  // Stagnation restart
  if (stagnationCounter > stagnationThreshold) {
    parent = generateInitialSolution(input);
    // Re-pin locked genes ...
  }

  // Early termination if feasible
  if (bestFitness.hardViolations === 0) break;
}
return { solution: bestSolution, fitness: bestFitness, ... };
}

```

A.5 LNS Auto-Fix for the Editor

The Large Neighbourhood Search routine from `localSearch.ts`, which identifies all clashing sessions and iteratively repairs them.

LISTING A.5: Full LNS Destroy-and-Repair (`localSearch.ts`)

```

export function runFullLNS(
  input: SolverInput,
  solution: Solution,
  maxAttempts: number = 1000,
): { solution: Solution; fitness: FitnessResult } | null {
  const baseFitness = evaluate(input, solution);
  if (baseFitness.hardViolations === 0) {
    return { solution, fitness: baseFitness };
  }

  // Find all session indices involved in violations
  let clashingIndices = findClashingIndices(input, solution);

```

```

let targets = Array.from(clashingIndices);
let bestSolution = solution.map(g => ({ ...g }));
let bestFitness = baseFitness;

for (let attempt = 0; attempt < maxAttempts; attempt++) {
  const candidate = bestSolution.map(g => ({ ...g }));
  // Pick a random clashing session
  const idx = targets[Math.floor(Math.random() * targets.length)];
  const session = input.sessions[idx];

  // Apply random mutation: relocate (50%), time shift (30%), room
  // (20%)
  const strategy = Math.random();
  if (strategy < 0.5)
    candidate[idx] = mutateRelocate(session, input.rooms);
  else if (strategy < 0.8)
    candidate[idx] = { ...candidate[idx],
      ...mutateTime(candidate[idx], session, 3.0) };
  else
    candidate[idx] = { ...candidate[idx],
      roomIndex: mutateRoom(session, input.rooms).roomIndex };

  const candidateFitness = evaluate(input, candidate);
  if (candidateFitness.total < bestFitness.total) {
    bestSolution = candidate;
    bestFitness = candidateFitness;
    if (bestFitness.hardViolations === 0) break;
    // Refresh targets after improvement
    clashingIndices = findClashingIndices(input, bestSolution);
    targets = Array.from(clashingIndices);
  }
}
return bestFitness.total < baseFitness.total
  ? { solution: bestSolution, fitness: bestFitness } : null;
}

```